# Proving Verifiability and Privacy with ProVerif

Vincent Cheval, Véronique Cortier, Alexandre Debant, Florian Moser

Workshop 25 years of ProVerif, June 3rd 2026

presented at CSF'23 and CSF'26





# Need for security proofs in evoting

Many protocols:

- Switzerland: Swiss Post, CHVote
- Ministère Affaires Étrangères (France): FLEP, new consulaires 2026
- Estonia, Australia, ...

Many attacks:

- Breaking Verifiability and Vote Privacy in CHVote. C., Debant, Gaudry 2025
- Reversing, Breaking, and Fixing the French Legislative Election E-Voting Protocol. Debant, Hirschi 2023
- Breaking the Encryption Scheme of the Moscow Internet Voting System. Gaudry, Golovnev 2020

# Need for security proofs in evoting

Many protocols:

- Switzerland: Swiss Post, CHVote
- Ministère Affaires Étrangères (France): FLEP, new consulaires 2026
- Estonia, Australia, ...

Many attacks:

- Breaking Verifiability and Vote Privacy in CHVote. C., Debant, Gaudry 2025
- Reversing, Breaking, and Fixing the French Legislative Election E-Voting Protocol. Debant, Hirschi 2023
- Breaking the Encryption Scheme of the Moscow Internet Voting System. Gaudry, Golovnev 2020

Swiss regulations

**5.1. Examining the cryptographic protocol**

| | |
|---|---|
| 5.1.1 | Examination criteria: The protocol must meet the security objective according to the trust assumptions in the abstract model in accordance with Section 4. In addition, a cryptographic and a symbolic proof must be provided. The proofs relating to cryptographic basic components may be provided according to generally accepted security assumptions (for example, the "random oracle model", "decisional Diffie-Hellman assumption", "Fiat-Shamir heuristic"). The protocol should be based if possible on existing and proven protocols. |

# Confidentiality of the votes

Vote privacy

*"No one should know how I voted"*

# Confidentiality of the votes

Vote privacy

*"No one should know how I voted"*

Better: Receipt-free / Coercion-resistant

*"No one should know how I voted,*
*even if I am willing to tell my vote! "*

- vote buying
- coercion

# Confidentiality of the votes

Vote privacy

*"No one should know how I voted"*

Better: Receipt-free / Coercion-resistant

*"No one should know how I voted, even if I am willing to tell my vote! "*

- vote buying
- coercion

Everlasting privacy: no one should know my vote, even when the cryptographic keys will be eventually broken.

# Verifiability

Individual Verifiability: a voter can check that

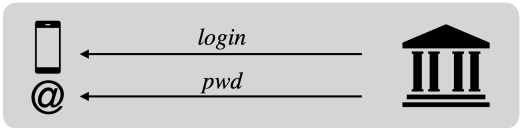
- cast as intended: their ballot contains their intended vote
- recorded as cast: their ballot is in the ballot box.

Universal Verifiability: everyone can check that

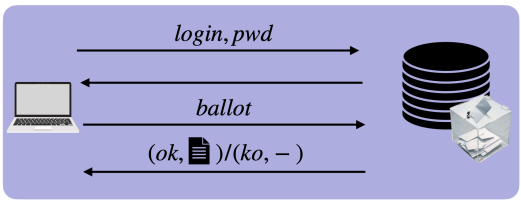
- tallied as recorded: the result corresponds to the ballot box.
- eligibility: ballots have been casted by legitimate voters.

You should verify the election,
not the system.

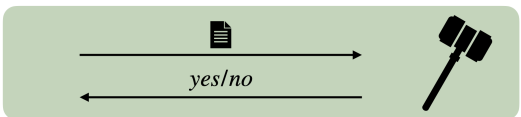
# Voting protocol - overview



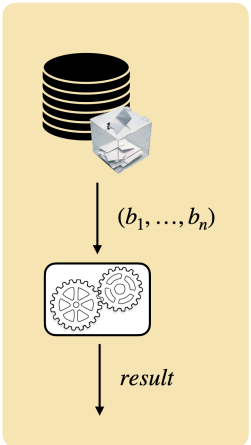

Setup phase




Voting phase



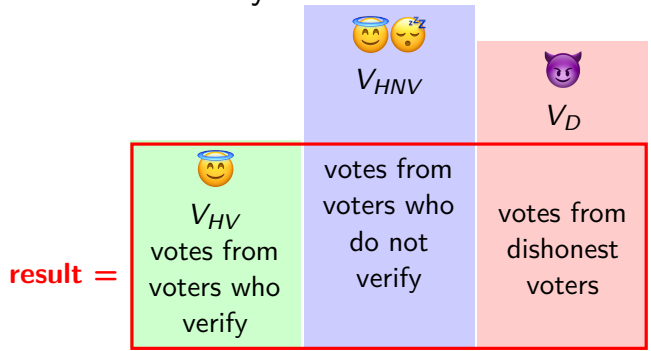

Verification phase




Tally phase

[slide borrowed from Alexandre Debant.]

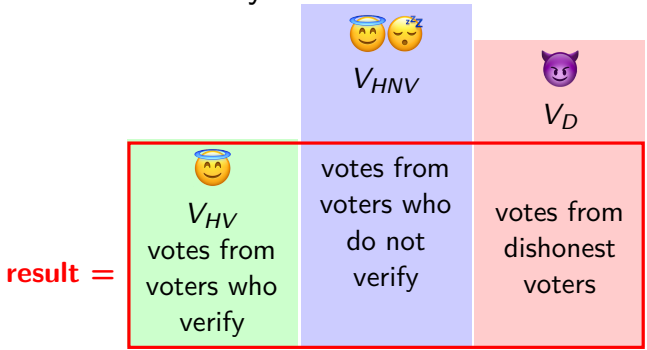
# End2End verifiability




**result =** $V_{HV} \uplus V'_{HNV} \uplus V'_D$ where

- $V'_{HNV} \subseteq V_{HNV}$
- $V'_D \subseteq$ Valid

# End2End verifiability




**result** = $V_{HV} \uplus V'_{HNV} \uplus V'_D$ where

- $V'_{HNV} \subseteq V_{HNV}$
- $V'_D \subseteq$ Valid

⚠ Hard to verify for tools

# Previous approaches: sufficient conditions

$\rightarrow$ check subproperties instead

**Theorem ([Cortier, Filipiak, Lallemand CSF'19, Baloglu, Bursuc, Mauw, Pang CSF'21])**

*eligibility* + *cast-as-intended* + *recorded-as-cast* + *tallied-as-recorded* + *no clash* $\Rightarrow$ *E2E Verifibility*

# Previous approaches: sufficient conditions

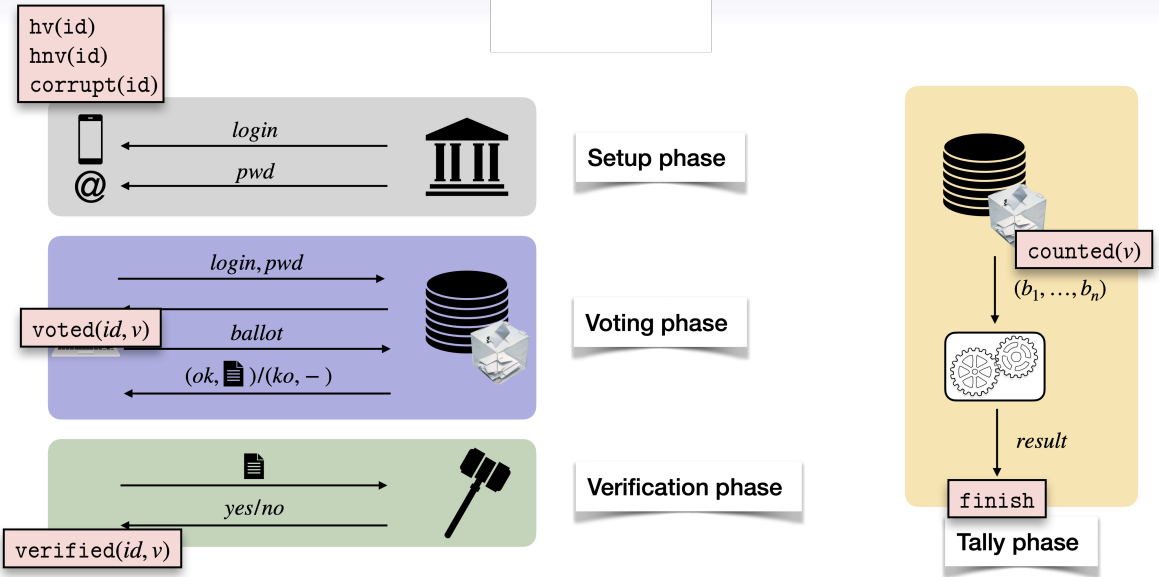
$\rightarrow$ check subproperties instead

**Theorem ([Cortier, Filipiak, Lallemand CSF'19, Baloglu, Bursuc, Mauw, Pang CSF'21])**

*eligibility* + *cast-as-intended* + *recorded-as-cast* + *tallied-as-recorded* + *no clash* $\Rightarrow$ *E2E Verifibility*

⚠ sufficient (but not tight) conditions

# Voting protocol - overview



# First contribution: exact characterization

**Theorem**

*E2E-verifiability* $\Leftrightarrow$ **query1** *and* **query2**

**query1**

$$\text{finish} \wedge \texttt{inj}-\text{verified}(z, x) \Rightarrow \texttt{inj}-\text{counted}(x)$$

Intuition: *individual verifability*

# First contribution: exact characterization

**Theorem**

*E2E-verifiability* $\Leftrightarrow$ **query1** *and* **query2**

**query1**

$$\text{finish} \wedge \texttt{inj}-\text{verified}(z, x) \Rightarrow \texttt{inj}-\text{counted}(x)$$

Intuition: *individual verifability*

**query2**

$$\begin{aligned} \text{finish} \wedge \texttt{inj}-\text{counted}(x) \quad \Rightarrow \quad & \texttt{inj}-\text{hv}(z) \wedge \text{verified}(z, x) \\ \vee \quad & \texttt{inj}-\text{hnv}(z) \wedge \text{voted}(z, x) \\ \vee \quad & \texttt{inj}-\text{corrupt}(z) \end{aligned}$$

Intuition: *extended universal verifiability*

# First contribution: exact characterization

**Theorem**

*E2E-verifiability* $\Leftrightarrow$ **query1** *and* **query2**

**query1** $\text{finish} \wedge \texttt{inj}-\text{verified}(z, x) \Rightarrow \texttt{inj}-\text{counted}(x)$

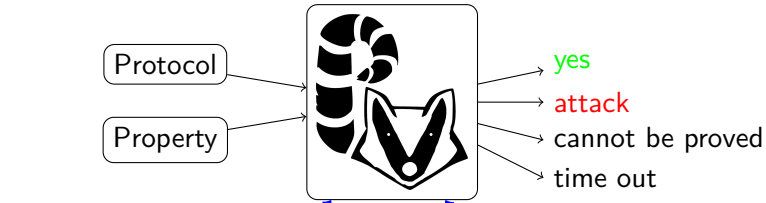
Intuition: *individual verifability*

**query2**
$$\text{finish} \wedge \texttt{inj}-\text{counted}(x) \Rightarrow \texttt{inj}-\text{hv}(z) \wedge \text{verified}(z, x) \vee \texttt{inj}-\text{hnv}(z) \wedge \text{voted}(z, x) \vee \texttt{inj}-\text{corrupt}(z)$$

Intuition: *extended universal verifiability*

Issue: make sure finish is executed only once all ballots are counted

# Second contribution: make it work in ProVerif




Framework for e-voting protocols

- 12 protocol specific processes
- 8 generic processes

Library for verifiability

- GSVerif-like axioms [CSF'18]
- 8 well-crafted lemmas

# Protocol specific processes

```
let Voting(e_id,v_idx,nb_vote,v,voting_data) =
  get election_key(=e_id,_,pkE) in
  let (pseudo,c_auth) = voting_data in
  new r_ctxt;
  let ctxt = aenc(pkE,v,r_ctxt) in
  event voted(e_id,v_idx,v);
  let b = (pseudo,ctxt) in
  out(c_pub, b); out(c_auth, b);

  out(res_voting(e_id,v_idx,nb_vote),b).
```

# Generic processes: Voter

Put together the user-defined processes

```
let Voter(e_id) =
  in(c_pub,v);
  if is_valid(v) then
  get voting_data(e_id,v_idx,v_data) in
  in(cell_voter(e_id,v_idx),nb_vote);
  Voting(e_id,v_idx,nb_vote+1,v,v_data) |
  in(res_voting(e_id,v_idx),res_data);
  in(c_pub, is_last);
  if is_last then
    Final_Check(e_id,v_idx,v,v_data,res_data,nb_vote+1)
  else
    out(cell_voter(e_id,v_index),nb_vote+1).
```

# Generic processes: Tally

Issue: we must read all ballots
$\rightarrow$ we simulate a loop

```
let Tally(e_id) =
  in(cell_tally(e_id),i);
  event Tally_Read(e_id,i)
  if i = 0 then event finish(e_id)
  else
    get public_identifier_id(=e_id,=i,ident) in
    in(cell_tally_last_vote(e_id,ident),x);
    if x = empty_ballot then out(cell_tally(e_id),i-1)
    else
      Decrypt_Ballot(e_id,i,ident,x) |
      in(res_decrypt(e_id,i),v);
      event Counted(e_id,v);
      event CountedExtended(e_id,v,i,ident);
      out(c_pub,v); out(cell_tally(e_id),i-1).
```

# A library for verifability

Axioms (correction guaranteed!)

- counter intervals
- term freshness

Generic lemmas

- to help with termination
- proved each time in ProVerif
- some using induction

# What about privacy?

How to state formally:

*"No one should know my vote (0 or 1)"?*

Idea 1: An attacker should not learn the value of my vote.

# What about privacy?

How to state formally:

*"No one should know my vote (0 or 1)"?*

Idea 1: An attacker should not learn the value of my vote.

But everyone knows 0 and 1!

# What about privacy?

How to state formally:

*"No one should know my vote (0 or 1)"?*

~~Idea 1: An attacker should not learn the value of my vote.~~

Idea 2: An attacker cannot see the difference when voters are different

$\text{Voter}(A, 0) \approx \text{Voter}(B, 0)$

# What about privacy?

How to state formally:

*"No one should know my vote (0 or 1)"?*

~~Idea 1: An attacker should not learn the value of my vote.~~

Idea 2: An attacker cannot see the difference when voters are different
$\text{Voter}(A, 0) \approx \text{Voter}(B, 0)$

Who voted might be public

# What about privacy?

How to state formally:

*"No one should know my vote (0 or 1)"?*

~~Idea 1: An attacker should not learn the value of my vote.~~

~~Idea 2: An attacker cannot see the difference when voters are different~~
~~$\text{Voter}(A, 0) \approx \text{Voter}(B, 0)$~~

Idea 3: An attacker cannot see the difference when I vote 0 or 1.

$$\text{Voter}(A, 0) \approx \text{Voter}(A, 1)$$

# What about privacy?

How to state formally:

*"No one should know my vote (0 or 1)"?*

~~Idea 1: An attacker should not learn the value of my vote.~~

~~Idea 2: An attacker cannot see the difference when voters are different~~
~~$\text{Voter}(A, 0) \approx \text{Voter}(B, 0)$~~

Idea 3: An attacker cannot see the difference when I vote 0 or 1.

$$\text{Voter}(A, 0) \approx \text{Voter}(A, 1)$$

- The attacker always sees the difference since the tally differs.
- Unanimity does break privacy.

# What about privacy?

How to state formally:

*"No one should know my vote (0 or 1)"?*

~~Idea 1: An attacker should not learn the value of my vote.~~

~~Idea 2: An attacker cannot see the difference when voters are different~~
~~$\text{Voter}(A, 0) \approx \text{Voter}(B, 0)$~~

~~Idea 3: An attacker cannot see the difference when I vote 0 or 1.~~

$$\text{Voter}(A, 0) \approx \text{Voter}(A, 1)$$

Idea 4: An attacker cannot see when votes are swapped.

$$\text{Voter}(A, 0) \mid \text{Voter}(B, 1) \approx \text{Voter}(A, 1) \mid \text{Voter}(B, 0)$$

S. Kremer & M. Ryan

# How to prove privacy?

# How to prove privacy?

Issue: our tally process breaks privacy!

```
let Tally(e_id) =
  in(cell_tally(e_id),i);
  event Tally_Read(e_id,i)
  if i = 0 then event finish(e_id)
  else
    get public_identifier_id(=e_id,=i,ident) in
    in(cell_tally_last_vote(e_id,ident),x);
    if x = empty_ballot then out(cell_tally(e_id),i-1)
    else
      Decrypt_Ballot(e_id,i,ident,x) |
      in(res_decrypt(e_id,i),v);
      event Counted(e_id,v);
      event CountedExtended(e_id,v,i,ident);
      out(c_pub,v); out(cell_tally(e_id),i-1).
```

# Let's fix the model

```
1 let Simplified_Tally(e_id) =
2   in(res_decrypt(e_id,1),v1);
3   in(res_decrypt(e_id,2),v2);
4
5   out(c_pub,v1); out(c_pub,v2)
```

```
1 let Simplified_Tally(e_id) =
2   in(res_decrypt(e_id,1),v1);
3   in(res_decrypt(e_id,2),v2);
4
5   out(c_pub,v1) | out(c_pub,v2)
```

✓ Model

✗ Proof

# Let's fix the model

```
1 let Simplified_Tally(e_id) =
2   in(res_decrypt(e_id,1),v1);
3   in(res_decrypt(e_id,2),v2);
4
5   out(c_pub,v1); out(c_pub,v2)
```

```
1 let Simplified_Tally(e_id) =
2   in(res_decrypt(e_id,1),v1);
3   in(res_decrypt(e_id,2),v2);
4
5   out(c_pub,v1) | out(c_pub,v2)
```

✓ Model
✗ Proof

⚠ ProVerif proves diff-equivalence.

- implies trace equivalence
- our change is ineffective

# Tally with swap

```
1 let Simplified_Tally(e_id) =
2   in(res_decrypt(e_id,1),v1);
3   in(res_decrypt(e_id,2),v2);
4
5   out(c_pub,v1) | out(c_pub,v2)
```

sound

Blanchet, Smyth 2018

```
1 let Simplified_Tally(e_id) =
2   in(res_decrypt(e_id,1),v1);
3   in(res_decrypt(e_id,2),v2);
4
5   out(c_pub,diff[v1, v2]) | out(c_pub,diff[v2, v1])
```

✓ Model

✓ Proof

✗ Termination

# Tally with swap

```
1 let Simplified_Tally(e_id) =
2   in(res_decrypt(e_id,1),v1);
3   in(res_decrypt(e_id,2),v2);
4
5   out(c_pub,v1) | out(c_pub,v2)
```

sound

Blanchet, Smyth 2018

```
1 let Simplified_Tally(e_id) =
2   in(res_decrypt(e_id,1),v1);
3   in(res_decrypt(e_id,2),v2);
4
5   out(c_pub,diff[v1, v2]) | out(c_pub,diff[v2, v1])
```

- ✓ Model
- ✓ Proof
- ✗ Termination

⚠ Not so easy to get right in our complex Tally process.

# Precision and termination: the CellBB lemma

```
1  mess(cell_BB(elec, i), diff[bL, bR]) ⟹
2    (bL = empty && bR = empty) (* either empty... *) ||
3    (
4      id = get_vid(i) &&
5
6      (* ...or correctly formed ballot in that cell... *)
7      (i, (cL, pL)) = bL &&
8      (i, (cR, pR)) = bR &&
9      (* ...encrypting some vote... *)
10     cL = encrypt(pk(elec), vL, rL) &&
11     cR = encrypt(pk(elec), vR, rR) &&
12     (* ...cryptographically bound to the voter... *)
13     pL = zkp_encrypt(pk(elec), (cL, id), (vL, rL)) &&
14     pR = zkp_encrypt(pk(elec), (cR, id), (vR, rR)) &&
15     (
16       (* ...has equal vote left/right... *)
17       (vL = vR && (vL = voteA || vL = voteB)) ||
18
19       (* ...except for honest voters 1 and 2 *)
20       ((id = 1) && (vL = voteA && vR = voteB)) ||
21       ((id = 2) && (vL = voteB && vR = voteA))
22     )
23   )
```

# Precision and termination: the CellBB lemma

```
1  mess(cell_BB(elec, i), diff[bL, bR]) ⟹
2    (bL = empty && bR = empty) (* either empty... *) ||
3    (
4      id = get_vid(i) &&
5
6      (* ...or correctly formed ballot in that cell... *)
7      (i, (cL, pL)) = bL &&
8      (i, (cR, pR)) = bR &&
9      (* ...encrypting some vote... *)
10     cL = encrypt(pk(elec), vL, rL) &&
11     cR = encrypt(pk(elec), vR, rR) &&
12     (* ...cryptographically bound to the voter... *)
13     pL = zkp_encrypt(pk(elec), (cL, id), (vL, rL)) &&
14     pR = zkp_encrypt(pk(elec), (cR, id), (vR, rR)) &&
15     (
16       (* ...has equal vote left/right... *)
17       (vL = vR && (vL = voteA || vL = voteB)) ||
18
19       (* ...except for honest voters 1 and 2 *)
20       ((id = 1) && (vL = voteA && vR = voteB)) ||
21       ((id = 2) && (vL = voteB && vR = voteA))
22     )
23   )
```

⚠ protocol specific

# Generic CellBB lemma

## Protocol specific

```
1 fun extract_vote(ballot): vote
2   reduc
3     let c = encrypt(pk(elec), v, _) in
4     let p = zkp_encrypt(pk(elec), (c, id), _(v, _)) in
5     extract_vote((c, p)) = v
6   [private]
```

## Framework defined

```
1 mess(cell_BB(elec, i), diff[bL, bR])@i ⟹
2   (bL = empty && bR = empty) ||
3     vL = extract_vote(bL) && vR = extract_vote(bR) &&
4     (
5       (attacker(diff[vL, vR])@j && i > j) ||
6       (i = 1) || (i = 2)
7     )
```

- ✓ Model
- ✓ Proof
- ✓ Termination

# Generic CellBB lemma

## Protocol specific

```
1 fun extract_vote(ballot): vote
2   reduc
3     let c = encrypt(pk(elec), v, _) in
4     let p = zkp_encrypt(pk(elec), (c, id), _(v, _)) in
5     extract_vote((c, p)) = v
6   [private]
```

## Framework defined

```
1 mess(cell_BB(elec, i), diff[bL, bR])@i ⟹
2   (bL = empty && bR = empty) ||
3     vL = extract_vote(bL) && vR = extract_vote(bR) &&
4     (
5       (attacker(diff[vL, vR])@j && i > j) ||
6       (i = 1) || (i = 2)
7     )
```

⚠ introduces destructors in lemma

- ✓ Model
- ✓ Proof
- ✓ Termination

# Generic CellBB lemma

## Protocol specific

```
1 fun extract_vote(ballot): vote
2   reduc
3     let c = encrypt(pk(elec), v, _) in
4     let p = zkp_encrypt(pk(elec), (c, id), _(v, _)) in
5     extract_vote((c, p)) = v
6   [private]
```

## Framework defined

```
1 mess(cell_BB(elec, i), diff[bL, bR])@i ⟹
2   (bL = empty && bR = empty) ||
3     vL = extract_vote(bL) && vR = extract_vote(bR) &&
4     (
5       (attacker(diff[vL, vR])@j && i > j) ||
6       (i = 1) || (i = 2)
7     )
```

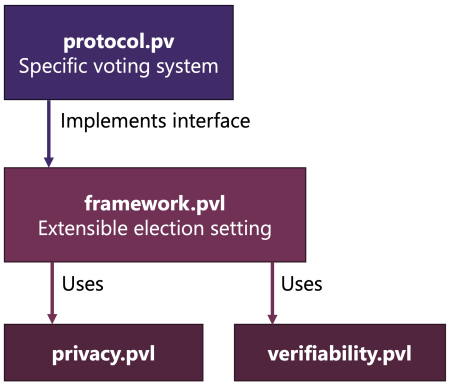
⚠ introduces destructors in lemma

- ✓ Model
- ✓ Proof
- ✓ Termination

Improved ProVerif lemmas

- destructor support
- letfun support

# Proof framework




[slide borrowed from Florian Moser]

Sound proof helpers

- GSVerif-line axioms (mostly for verif)
- custom lemmas / cellBB
- no-select statements

- `privacy.pl` implements cellBB, privacy definitions & swap
- `privacy.pl` implements verifiability annotations for the (many) lemmas

# Case studies

| | Verifiability | | Privacy | |
|---|---|---|---|---|
| **Running example** Helios-like | | | | |
| with honest decryption 1 | ✓ | 6s | ✓ | 11s |
| with honest decryption 2 | ✓ | 6s | ✓ | 7s |
| with honest decryption 3 | ✓ | 143s | ✓ | 5s |
| **Belenios** | | | | |
| with honest server | ✓ | 13s | ✓ | 20s |
| with honest registrar | ✓ | 13s | ✓ | 13s |
| with honest server & registrar | ✓ | 13s | ✓ | 16s |
| **Swiss Post Protocol** | | | | |
| with honest CCR1 & CCM1 | ✓ | 1092s | ✓ | 18s |
| with honest CCR1 & CCM2 | ✓ | 1089s | ✓ | 11s |
| with honest CCR2 & CCM1 | ✓ | 1093s | ✓ | 18s |
| with honest CCR2 & CCM2 | ✓ | 1091s | ✓ | 11s |
| **Short Voting Codes** | ✓ | 851s | ✓ | 1228s |
| CHVote | ✓ | | open issues | |
| Vote & Check | ✓ | | not supported | |

# Conclusion

- Tally phase finally modeled!
- rather “plug and play” for existing ProVerif files
- make use of recent features of ProVerif (counters, lemmas, ...)

## Future work

- extend to protocols with trackers (eg Select, Selene, Vote & Check)
- better mixnets with AC symbols?
- could it apply to other tools such as Tamarin?